

Handling Legacy Fortran Code

Rohit Goswami · 2023-04-27 · fortran · f2py

A short pseudo-tutorial post on working with inherited Fortran.

Background

In spite of the many claims by upper management (e.g. at LANL) regarding the death of Fortran, most people with even a passing interest in academic research within STEM fields will interact with Fortran code, and the community has only been improving with time Kedward et al. (2022). Rewriting **is not** typically an option, or a good use of time. This set of posts should be applicable to any blackbox legacy fortran code, though the specific example here is one from my own field (computational chemistry).

The Code

As an exemplary example, we will consider an implicitly typed Fortran-77 program for generating the potential energy surface (and forces) of a system containing Copper and Hydrogen atoms. The group I'm associated with has a rather nice Embedded Atom Model parameterized for this particular system [^fn:1]. The source can be obtained from here:

```
git clone https://github.com/TheochemUI/fortran_cuh2_src
```

Since it was initially meant to work with `con` files, the salient points of the implementation are:

- `main.f90` uses file I/O to read `tmp.con` and write to `energy.out` and `forces.out`, and signals that it is ready by writing `1` to `eam.tmp`
- `eamroutines.f90` reads in `pot.par`, which has parameters used in a space separated file:

```
6.10000  6.30000
.000000E+00 .000000E+00 .000000E+00
2862.30    79.5013    86.1495
3.51236    2.47961    4.21154
-109.107   -107.555    15355.5
....
```

Additionally the code is in FORTRAN 77, and uses features no longer in vogue including implicit typing and `common` blocks.

Initial Cleanup

Rather than hard-coding a path (`pot.par`) it is best to store these values in the source code itself.

```
potpar = np.loadtxt("pot.par", skiprows=1) # first row has 2 values
for idx, val in enumerate(aa, start=1):
    print(f"potpar({idx}, 1) = {val[0]}\n"
          f"potpar({idx}, 2) = {val[1]}\n"
          f"potpar({idx}, 3) = {val[2]}")
```

There are many more possible optimizations which can be done on the Fortran side, but this is the bare minimum we need to move on. The state of the project can be seen in [1461cdf](#). The `main.f90` issues regarding the file I/O can be handled from different languages.

Python Usage

When confronted by a Fortran 77 code, or any Fortran code in general, it is never a bad idea to start out with `f2py`. In this instance, we would like to call the `force_eam` function without all the file I/O:

```
f2py -c -m cuh2 eamroutines.f90
```

Now we can use this from Python:

```
import numpy as np
import cuh2 # Compiled module
R = [0.63940268750835, 0.90484742551374, 6.97516498544584, 3.19652040936288, 0.90417430354811,
6.97547796369474, 8.98363230369760, 9.94703496017833, 7.83556854923689, 7.64080177576300, 9.94703114803832,
7.83556986121272]
box = np.array([15.34, 21.702, 100])
natms = np.array([2, 2])
u = np.zeros(1)
F = np.zeros_like(R)
cuh2.force_eam(natms, box, R.ravel(), F.ravel(), u) # Call!
print(u)
[-2.71140933]
```

This works very well, and in-fact, if this use case (flexibly calling a function) is all that is needed, then nothing more needs to be done. `f2py` also generates correct signatures, and doc-strings:

```
??cuh2.force_eam
force_eam(natms,box,r,f,u,[ndim])

Wrapper for ``force_eam``.

Parameters
-----
natms : input rank-1 array('i') with bounds (2)
box : input rank-1 array('d') with bounds (3)
r : input rank-1 array('d') with bounds (ndim)
f : input rank-1 array('d') with bounds (ndim)
u : input rank-1 array('d') with bounds (1)

Other Parameters
-----
ndim : input int, optional
      Default: shape(r, 0)
```

The resulting compiled extensions can also be published to PyPI as SciPy does.

ISO_C_BINDING wrappers for C++ and C

An `iso_c_binding` is essential for rational interfacing to C++ or other languages with a well-defined C-API. Since here there is only one function which needs to be called it can be as simple as:

```

module eam_wrap
  use iso_c_binding, only: c_double, c_int
  implicit none
contains
  subroutine c_force_eam (natms, ndim, box, R, F, U) bind(c)
    integer(c_int), intent(in) :: natms(2)
    integer(c_int), value, intent(in) :: ndim
    real(c_double), intent(in) :: box(3)
    real(c_double), intent(inout) :: U(1), R(ndim), F(ndim)
    call force_eam(natms, ndim, box, R, F, U)
  end subroutine c_force_eam
end module eam_wrap

```

Which, can be mapped to the following function in any `C` header file:

```

extern void c_force_eam(int *natms, int ndim,
                      double *box, double *R,
                      double *F, double *U);

```

We can always check that the right functions are where they should be:

```

gfortran -c eam_isoc.f90 eamroutines.f90
nm eam_isoc.o
0000000000000000 T c_force_eam
                  U force_eam_
nm eamroutines.o
0000000000000000 c C counters_
0000000000000050 C cutoff_values_
0000000000000065 c8 C eam_
00000000000000130 C eamdblexp_
0000000000000025 e8 T eamfcu_
00000000000000249 b T eamfh_
0000000000000000 T eamh2cu_
0000000000000029 ef T eamphicucu_
0000000000000028 f8 T eamphihcu_
0000000000000028 01 T eamphihh_
0000000000000023 31 T eamrhocu_
0000000000000021 c7 T eamrhoh_
                  U exp
0000000000000032 10 T force_eam_
                  U _gfortran_stop_string
                  U _gfortran_st_write
                  U _gfortran_st_write_done
                  U _gfortran_transfer_character_write
                  U _gfortran_transfer_integer_write
                  U _gfortran_transfer_real_write
000000000000002a e6 T potinit_
                  U __powidf2
                  U __stack_chk_fail

```

MATLAB Interfaces

Nominally, MATLAB has a [Fortran API](#). However, it isn't great. The `C++` API is “modern” to the extent of of `C++11`, which is practically useless. So the `C` [MEX API](#) is the best way forward here, though **input arrays need to be transposed** if they are passed to Fortran routines.

Conclusions

This was a first pass at what to do when confronted with Fortran code, short of running away, or rewriting entirely. Follow up posts handle integrating into larger C++ codes, or distributing said libraries with `pip` [^fn:2].

References

Kedward, Laurence J., Bálint Aradi, Ondřej Čertík, Milan Curcic, Sebastian Ehlert, Philipp Engel, Rohit Goswami, et al. 2022. "The State of Fortran." **Computing in Science & Engineering** 24 (2): 63--72. <<https://doi.org/10.1109/MCSE.2022.3159862>>.

qualitatively correct. It captures the fact that when the slab is fixed, there are effectively only two degrees of freedom, the distance between the hydrogen atoms, and the distance to the slab.

are largely build system related, and require no changes to the Fortran code other than a compatible ``iso_c_binding`` wrapper so they are extracted away.